Program Control
(C Programming Language)

Dr. Thien Huynh-The

Dept. Comp. Commun. Eng.

HCMC Technology and Education

Content

- Introduction
- Repetition Essentials
- Counter-Controlled Repetition
- `for` Repetition Statement
- `for` Statement: Notes and Observations
- Examples Using the for Statement
- `switch` Multiple-Selection Statement
- `do...while` Repetition Statement
- `break and continue` Statements
- Logical Operators
- Confusing Equality (==) and Assignment (=) Operators
- Structured Programming Summary

Introduction

- This chapter introduces
  - Additional repetition control structures
    - `for`
    - `do…while`
  - `switch` multiple selection statement
  - `break` statement
    - Used for exiting immediately and rapidly from certain control structures
  - `continue` statement
    - Used for skipping the remainder of the body of a repetition structure and proceeding with the next iteration of the loop

Repetition Essentials

- Loop
  - Group of instructions computer executes repeatedly while some condition remains true
- Counter-controlled repetition
  - Definite repetition: know how many times loop will execute
  - Control variable used to count repetitions
- Sentinel-controlled repetition
  - Indefinite repetition
  - Used when number of repetitions not known
  - Sentinel value indicates "end of data"

Counter-Controlled Repetition

- Counter-controlled repetition requires
  - The name of a control variable (or loop counter)
  - The initial value of the control variable
  - An increment (or decrement) by which the control variable is modified each time through the loop
  - A condition that tests for the final value of the control variable (i.e., whether looping should continue)
- Example:

```
int counter = 1;                    // initialization
while ( counter <= 10 ) {           // repetition condition
   printf( "%d \n", counter );
   ++counter;                       // increment
}
```

Counter-Controlled Repetition

- C Programmers would make the program more concise

```
Initialize counter to 0

while ( ++counter <= 10 )
    printf( “%d \n”, counter );
```

- Note:
  - Controlling counting loops with floating-point variables may result in imprecise counter values
  - Avoid using more than three levels of nesting.

for Iteration Statement

```c
// Fig. 4.2: fig04_02.c
// Counter-controlled iteration with the for statement.
#include <stdio.h>

int main(void)
{
   // initialization, iteration condition, and increment
   //  are all included in the for statement header.
   for (unsigned int counter = 1; counter <= 10; ++counter) {
      printf("%u\n", counter);
   }
}
```

Fig. 4.2 | Counter-controlled iteration with the `for` statement.

for Iteration Statement

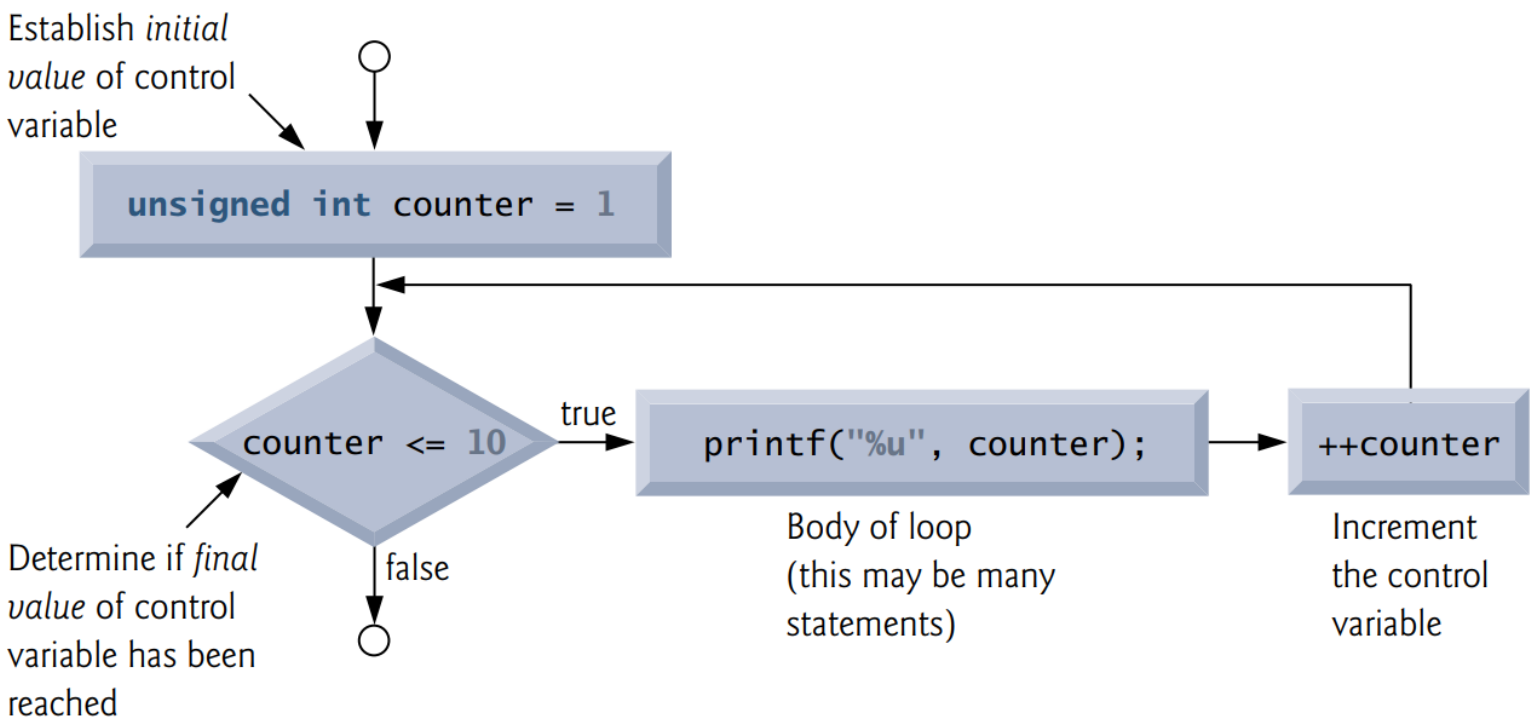


Fig. 4.4 | Flowcharting a typical `for` iteration statement.

for Iteration Statement

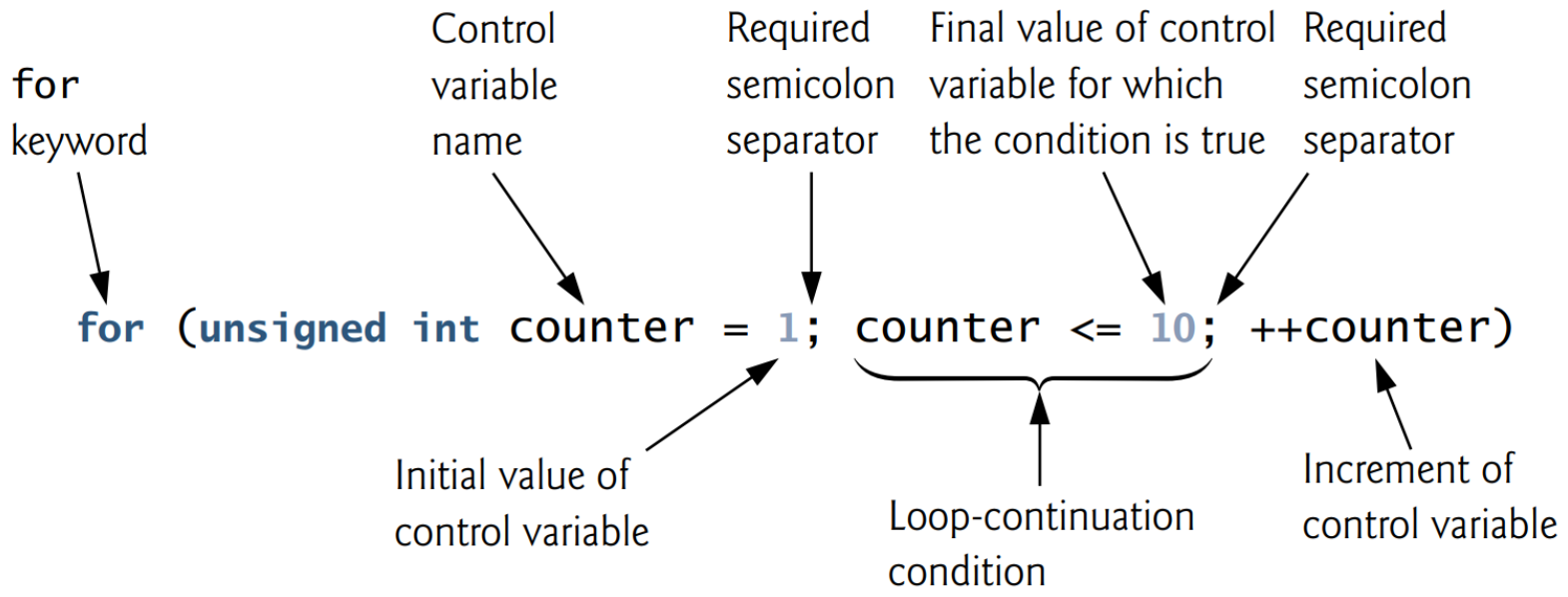


Fig. 4.3 | for statement header components.

- Using the final value in the condition of a while or for statement and using the <= relational operator will help avoid off-by-one errors.
- The loop-continuation condition should be counter <= 10 rather than ~~counter < 11~~ or ~~counter < 10~~.

for Iteration Statement

- Format when using for loops

```
for ( initialization; loopContinuationTest; increment )
  statement;
```

- Example (print the integers from one to ten):

```
for( int counter = 1; counter <= 10; counter++ )
   printf( "%d \n", counter );
```

- `for` loops can usually be rewritten as `while` loops:

```
initialization;
while ( loopContinuationTest ) {
   statement;
   increment;}
```

- Initialization and increment
  - Can be comma-separated lists. Example (what does a program print?):

```
for (int i = 0, j = 0;  j + i <= 10; j++, i++)
   printf( "%d \n", j + i );
```

`for` Iteration Statement

- Using commas instead of semicolons in a for header is a syntax error.

```
for( int counter = 1, counter <= 10, counter++ )
```

- Placing a semicolon immediately to the right of a for header makes the body of that for statement an empty statement. This is normally a logic error.

```
for( int counter = 1; counter <= 10; counter++ );
```

Exercises

- Bài 01: Viết chtrình tính tổng các số nguyên từ 1 đến 10
- Bài 02: Viết chtrình tính tổng các số lẻ từ 50 đến 100
- Bài 03: Viết chtrình nhập vào số a lớn hơn hoặc bằng 2, tính giai thừa của a
- Bài 04: Viết chtrình tính tiền lãi gửi tiết kiệm theo năm với lãi suất 8%/năm (đáo hạn lãi nhập vốn)
- Bài 05: Viết chtrình nhập x và n và tính giá trị x^n
- Bài 06: Viết chtrình nhập n và tính

$$1 + \frac{1}{2^1 2!} + \frac{1}{2^2 3!} + \cdots + \frac{1}{2^n (n+1)!}$$

Yêu cầu sử dụng vòng lặp `for`

`for` Iteration Statement

- Arithmetic expressions
  - Initialization, loop-continuation, and increment can contain arithmetic expressions. If x equals 2 and y equals 10

```
for ( j = x; j <= 4 * x * y; j += y / x )
```

is equivalent to

```
for ( j = 2; j <= 80; j += 5 )
```

- Notes about the `for` statement:
  - "Increment" may be negative (decrement)
  - If the loop continuation condition is initially false
    - The body of the `for` statement is not performed
    - Control proceeds with the next statement after the for statement
  - Control variable
  - Often printed or used inside for body, but not necessary

for Iteration Statement

```c
// Fig. 4.5: fig04_05.c
// Summation with for.
#include <stdio.h>

int main(void)
{
   unsigned int sum = 0; // initialize sum

   for (unsigned int number = 2; number <= 100; number += 2) {
      sum += number; // add number to sum
   }

   printf("Sum is %u\n", sum);
}
```

```
Sum is 2550
```

- Although the value of the control variable can be changed in the body of a for loop, this can lead to subtle errors. It is best not to change it.
- Limit the size of control-statement headers to a single line if possible.

Exercise

A person invests 1000 usd in a savings account yielding 5% interest. Assuming that all interest is left on deposit in the account, calculate and print the amount of money in the account at the end of each year for 10 years. Use the following formula for determining these amounts:

$$a = p(1+r)^n$$

where

p is th original amount invested

r is the annual interest rate

n is the number of years

a is the amount on deposit at the end of the nth year

```c
#include <stdio.h>
#include <math.h>
    int main(){
    double amount;
    double principal = 1000.0;
    double rate = 0.05;
    int year;
    printf("%4s%21s \n", "Year", "Amount on deposit");
    for (year = 1; year <= 10; year++) {
        amount = principal * pow(1.0 + rate, year);
        printf("%4d%21.2f\n", year, amount);
    }
    return 0;
}
```

```
Year    Amount on deposit
   1              1050.00
   2              1102.50
   3              1157.63
   4              1215.51
   5              1276.28
   6              1340.10
   7              1407.10
   8              1477.46
   9              1551.33
  10              1628.89
```

`switch` Multiple-Selection Statement

- Switch: useful when a variable or expression is tested for all the values it can assume and different actions are taken
- Format: series of case labels and an optional default case

```
switch ( value ){
case '1’:
       actions
case '2’:
       actions
default:
       actions
}
break; //exits from statement
```

switch Multiple-Selection Statement

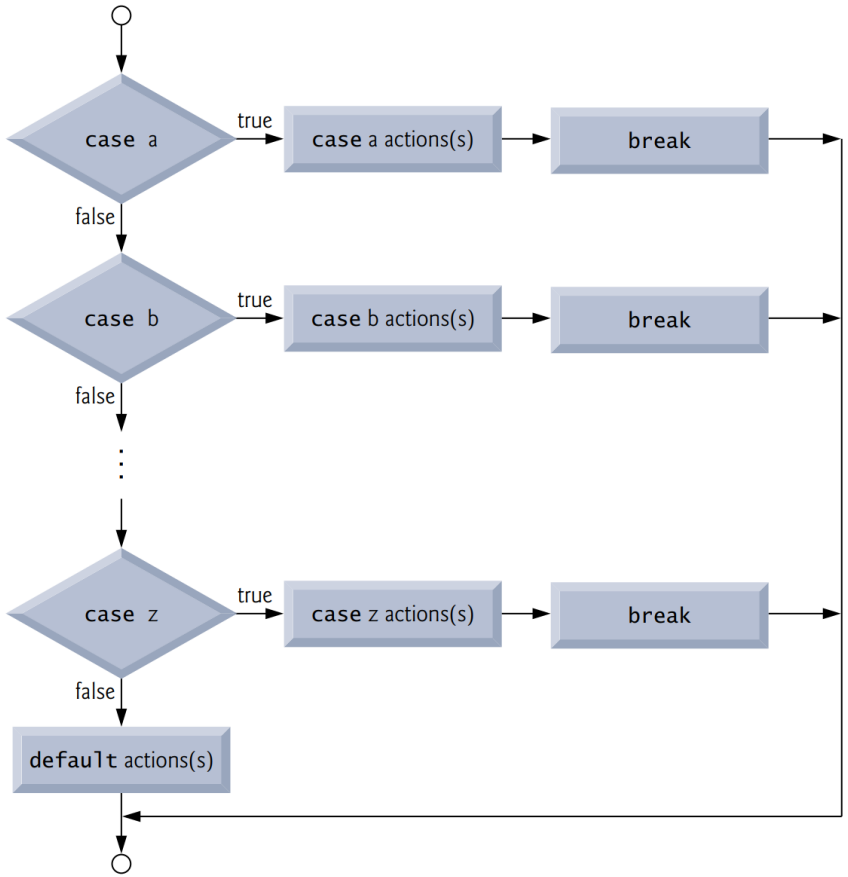


Fig. 4.8 | switch multiple-selection statement with breaks.

Exercises

- Viết chtrình C theo yêu cầu sau đây dùng lệnh switch
  - Nhập vào toán tử muốn thực hiện (ví dụ, +, -, *, /)
  - Nhập vào 2 biến a và b muốn thực hiện phép toán
  - In ra kết quả của toán tử với 2 biến

```
nhap vao toan tu (+, -, *, /): +
nhap vao gia tri 2 so: 45.7 14.2
45.7 + 14.2 = 59.9
```

```
char toantu;
switch(toantu){
    case '+':
```

Exercises

- Viết chtrình C theo yêu cầu sau đây dùng lệnh switch
  - Nhập vào các chữ cái (ứng với xếp loại)
  - Đếm có bao nhiêu chữ cái được nhập (chỉ tính a, b, c, d và không phân biệt viết hoa hay viết thường)
    - Nếu nhập chữ cái khác a,b,c,d thì báo sai và yêu cầu nhập lại
    - Không muốn nhập nữa thì nhấn tổ hợp phím Ctrl+Z
  - In ra có bao nhiêu lần nhập các chữ cái a, b, c, d

```
puts("nhap vao xep loai (A-D): ");
puts("nhan to hop phim Ctrl+Z neu muon ket thuc!");
int loai;
while ((loai=getchar()) != EOF){
```

Example

```c
// Fig. 4.7: fig04_07.c
// Counting letter grades with switch.
#include <stdio.h>

int main(void)
{
   unsigned int aCount = 0;
   unsigned int bCount = 0;
   unsigned int cCount = 0;
   unsigned int dCount = 0;
   unsigned int fCount = 0;

   puts("Enter the letter grades.");
   puts("Enter the EOF character to end input.");
   int grade; // one grade

   // loop until user types end-of-file key sequence
   while ((grade = getchar()) != EOF) {

      // determine which grade was input
      switch (grade) { // switch nested in while

         case 'A': // grade was uppercase A
         case 'a': // or lowercase a
            ++aCount;
            break; // necessary to exit switch
```

getchar function (from <stdio.h>) reads one character from the keyboard and stores that character in the integer variable grade.

EOF stands for "end of file;" this character varies from system to system

switch statement checks each of its nested cases for a match

break statement makes program skip to end of switch

Example

```
27
28          case 'B': // grade was uppercase B
29          case 'b': // or lowercase b
30             ++bCount;
31             break;
32
33          case 'C': // grade was uppercase C
34          case 'c': // or lowercase c
35             ++cCount;
36             break;
37
38          case 'D': // grade was uppercase D
39          case 'd': // or lowercase d
40             ++dCount;
41             break;
42
43          case 'F': // grade was uppercase F
44          case 'f': // or lowercase f
45             ++fCount;
46             break;
47
48          case '\n': // ignore newlines,
49          case '\t': // tabs,
50          case ' ': // and spaces in input
51             break;
52
53          default: // catch all other characters
54             printf("%s", "Incorrect letter grade entered.");
55             puts(" Enter a new grade.");
56             break; // optional; will exit switch anyway
57       }
58    } // end while
59
60    // output summary of results
61    puts("\nTotals for each letter grade are:");
62    printf("A: %u\n", aCount);
63    printf("B: %u\n", bCount);
64    printf("C: %u\n", cCount);
65    printf("D: %u\n", dCount);
66    printf("F: %u\n", fCount);
67 }
```

default case occurs if none of the cases are matched

Example

```
Enter the letter grades.
Enter the EOF character to end input.
a
b
c
C
A
d
f
C
E
Incorrect letter grade entered. Enter a new grade.
D
A
b
^Z
```

Not all systems display a representation of the EOF character

```
Totals for each letter grade are:
A: 3
B: 2
C: 3
D: 2
F: 1
```

`switch` Multiple-Selection Statement

Notices:

- Testing for the symbolic constant EOF rather than –1 makes programs more portable. Thus, EOF could have different values on different systems.
- Forgetting a break statement when one is needed in a switch statement is a logic error.
- Provide a default case in switch statements.
- Place the default clause last.
- The break statement is not required.
- Remember to provide processing capabilities for newline

do…while Repetition Statement

- The do…while repetition statement
  - Similar to the while structure
  - Condition for repetition only tested after the body of the loop is performed
  - All actions are performed at least once
- Format:

```
do {
  statement;
} while ( condition );
```

- Example (letting counter = 1):

```
do {
   printf( "%d  ", counter );
} while (++counter <= 10);
```

Prints the integers from 1 to 10

`do…while` Repetition Statement

- Some programmers always include braces in a `do...while` statement even if the braces are not necessary.
- Infinite loops are caused when the loop-continuation condition in a `while`, `for` or `do...while` statement never becomes false. To prevent this, make sure there is not a semicolon immediately after the header of a while or for statement.
- In a counter-controlled loop, make sure the control variable is incremented (or decremented) in the loop. In a sentinel-controlled loop, make sure the sentinel value is eventually input.

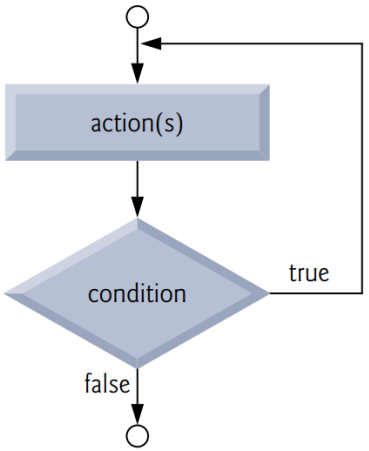
do…while Repetition Statement

```c
// Fig. 4.9: fig04_09.c
// Using the do...while iteration statement.
#include <stdio.h>

int main(void)
{
   unsigned int counter = 1; // initialize counter

   do {
      printf("%u  ", counter);
   } while (++counter <= 10);
}
```

increments counter then checks if it is less than or equal to 10




```
1  2  3  4  5  6  7  8  9  10
```

Exercise

- Viết chtrình sử dụng vòng lặp `do...while`
  - Nhập điểm cho sinh viên
  - Dừng việc nhập điểm nếu số sv dưới trung bình từ 5 bạn trở lên
  - Sau khi dừng nhập điểm, in ra số sv trên trung bình cùng điểm trung bình của các sv đó

`break` and `continue` Statements

- `break` statement
  - Causes immediate exit from a `while`, `for`, `do…while` or `switch` statement
  - Program execution continues with the first statement after the structure
  - Common uses of the break statement
    - Escape early from a loop
    - Skip the remainder of a switch statement

break and continue Statements

```c
// Fig. 4.11: fig04_11.c
// Using the break statement in a for statement.
#include <stdio.h>

int main(void)
{
   unsigned int x; // declared here so it can be used after loop

   // loop 10 times
   for (x = 1; x <= 10; ++x) {

      // if x is 5, terminate loop
      if (x == 5) {
         break; // break loop only if x is 5
      }

      printf("%u ", x);
   }

   printf("\nBroke out of loop at x == %u\n", x);
}
```

break immediately ends for loop

```
1 2 3 4
Broke out of loop at x == 5
```

`break` and `continue` Statements

- Continue statement
  - Skips the remaining statements in the body of a `while`, `for` or `do…while` statement
    - Proceeds with the next iteration of the loop
  - `while` and `do…while`
    - Loop-continuation test is evaluated immediately after the continue statement is executed
  - `for`
    - Increment expression is executed, then the loop-continuation test is evaluated

break and continue Statements

```c
// Fig. 4.12: fig04_12.c
// Using the continue statement in a for statement.
#include <stdio.h>

int main(void)
{
   // loop 10 times
   for (unsigned int x = 1; x <= 10; ++x) {

      // if x is 5, continue with next iteration of loop
      if (x == 5) {
         continue; // skip remaining code in loop body
      }

      printf("%u ", x);
   }

   puts("\nUsed continue to skip printing the value 5");
}
```

continue skips to end of for loop and performs next iteration

```
1 2 3 4 6 7 8 9 10
Used continue to skip printing the value 5
```

Exercise

- Viết chtrình nhập điểm cho sv (không biết số lượng sv)
  - Thực hiện việc nhập và đọc điểm cho đến khi nào có 2 điểm 10 đầu tiên thì dừng lại
  - In ra số lượng sv có điểm từ 8 trở lên

`break` and `continue` Statements

- Some programmers feel that break and continue violate the norms of structured programming.
- The break and continue statements, when used properly, perform faster than the corresponding structured techniques that we will soon learn.
- There is a tension between achieving quality software engineering and achieving the best-performing software. Often one of these goals is achieved at the expense of the other.

Logical Operations

- && (logical AND) – Returns true if both conditions are true
- || (logical OR) – Returns true if either of its conditions are true
- ! (logical NOT, logical negation) – Reverses the truth/falsity of its condition, unary operator, has one operand
- Useful as conditions in loops

Expression	Result
true && false	false
true || false	true
!false	true

Logical Operators

expression1	expression2	expression1 && expression2
0	0	0
0	nonzero	0
nonzero	0	0
nonzero	nonzero	1

Fig. 4.13 | Truth table for the logical AND (&&) operator.

expression1	expression2	expression1 || expression2
0	0	0
0	nonzero	1
nonzero	0	1
nonzero	nonzero	1

Fig. 4.14 | Truth table for the logical OR (||) operator.

Logical Operators

- In expressions using operator &&, make the condition that is most likely to be false the leftmost condition.
- In expressions using operator ||, make the condition that is most likely to be true the leftmost condition. This can reduce a program's execution time.

expression	!expression
0	1
nonzero	0

Fig. 4.15 | Truth table for operator ! (logical negation).

Operator Precedence

Operators	Associativity	Type
++ *(postfix)* -- *(postfix)*	right to left	postfix
+ - ! ++ *(prefix)* -- *(prefix)* (*type*)	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
&&	left to right	logical AND
||	left to right	logical OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment
,	left to right	comma

Fig. 4.16 | Operator precedence and associativity.

Confusing (==) and (=) Operators

- Dangerous error
  - Does not ordinarily cause syntax errors
  - Any expression that produces a value can be used in control structures
  - Nonzero values are true, zero values are false
  - Example using ==:

```
if ( payCode == 4 )
   printf( "You get a bonus!\n" );
```

- Checks payCode, if it is 4 then a bonus is awarded
- Example, replacing == with =:

```
if ( payCode = 4 )
   printf( "You get a bonus!\n" );
```

→ This sets payCode to 4

4 is nonzero, so expression is true, and bonus awarded no matter what the payCode was → Logic error, not a syntax error

Confusing (==) and (=) Operators

- lvalues
  - Expressions that can appear on the left side of an equation
  - Their values can be changed, such as variable names
    - `x = 4;`
- rvalues
  - Expressions that can only appear on the right side of an equation
  - Constants, such as numbers
    - Cannot write `4 = x;`
    - Must write `x = 4;`
  - lvalues can be used as rvalues, but not vice versa
    - `y = x;`

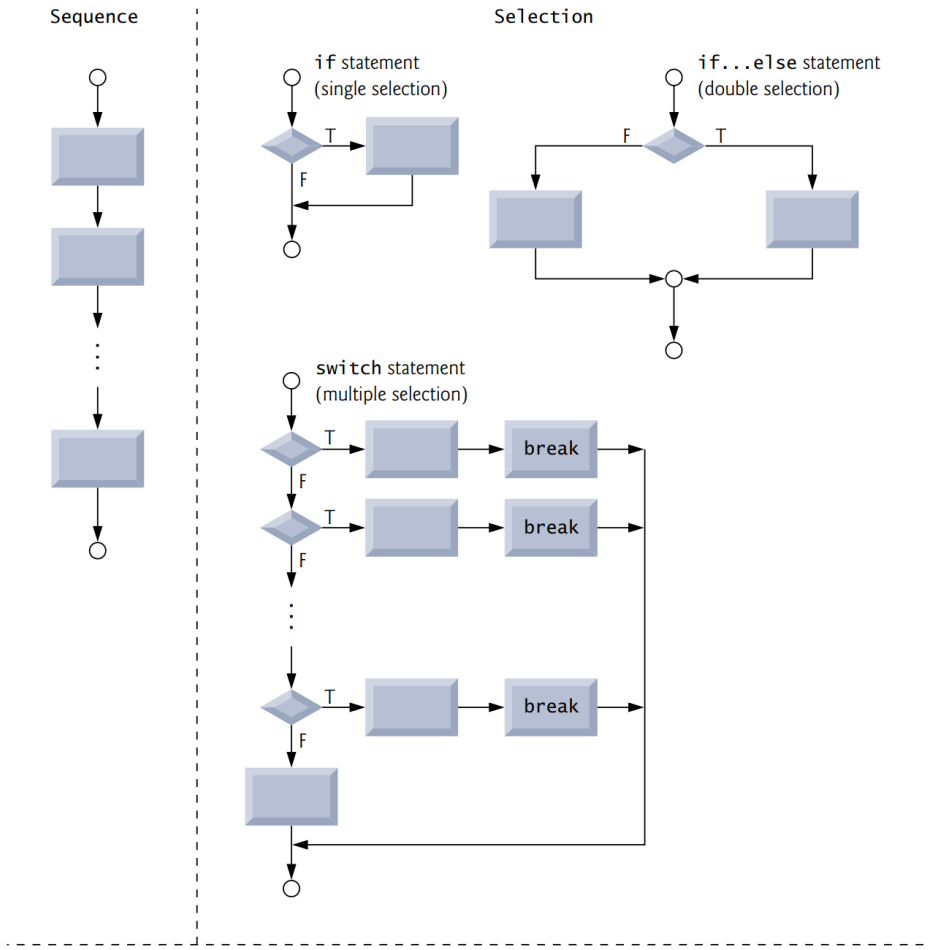
Confusing (==) and (=) Operators

- When an equality expression has a variable and a constant, as in `x == 1`, some programmers prefer to write the expression with the constant on the left and the variable name on the right

  (e.g. `1 == x` as protection against the logic error that occurs when you accidentally replace operator == with =).

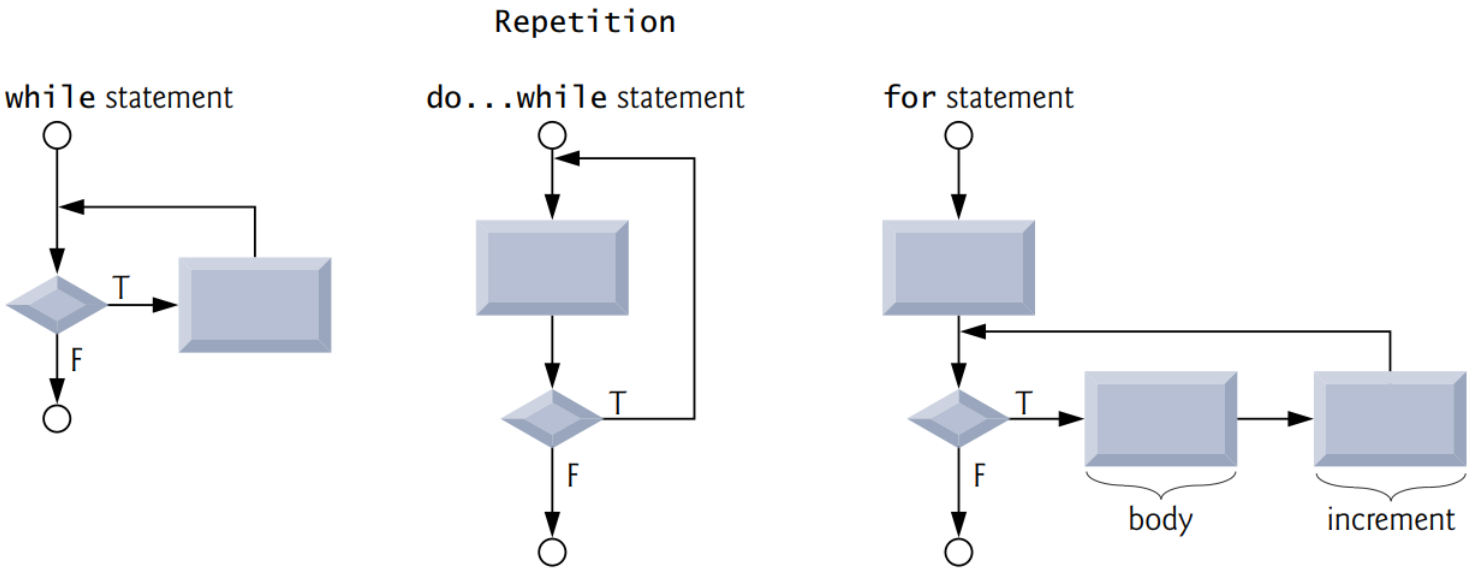
- After you write a program, text search it for every = and check that it is being used properly.

Summary


Sequence
Selection
if statement
(single selection)
T
F
if...else statement
(double selection)
F
T
switch statement
(multiple selection)
T
F
break
T
F
break
T
F
break

Summary

- Structured programming
  - Easier than unstructured programs to understand, test, debug and, modify programs



Summary

Rules for forming structured programs
1. Begin with the "simplest flowchart" (Fig. 4.19). 2. ("Stacking" rule) Any rectangle (action) can be replaced by *two* rectangles (actions) in sequence. 3. ("Nesting" rule) Any rectangle (action) can be replaced by *any* control statement (sequence, `if`, `if...else`, `switch`, `while`, `do...while` or `for`). 4. Rules 2 and 3 may be applied as often as you like and in *any* order.

Fig. 4.18 | Rules for forming structured programs.

Summary

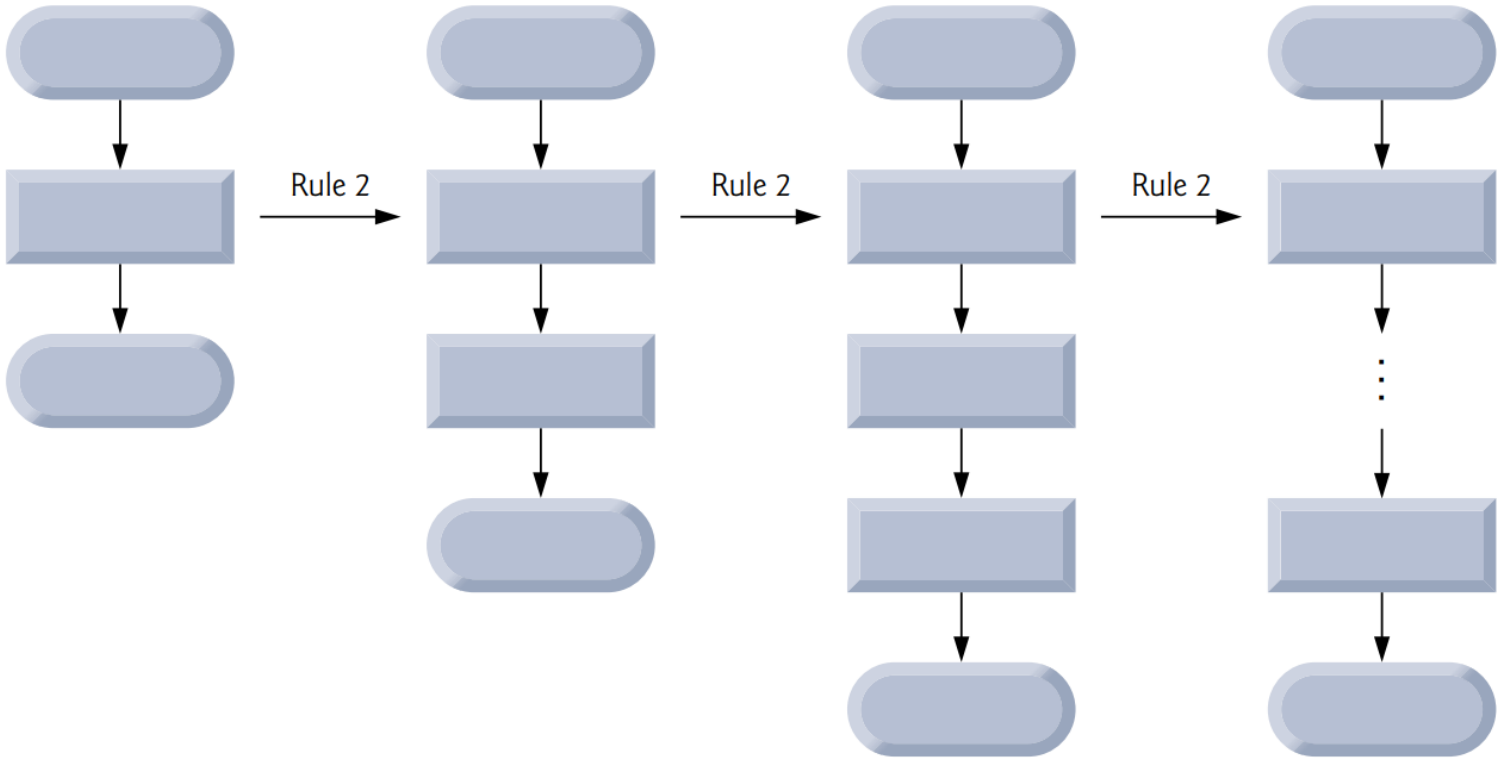


Fig. 4.20 | Repeatedly applying Rule 2 of Fig. 4.18 to the simplest flowchart.

Summary

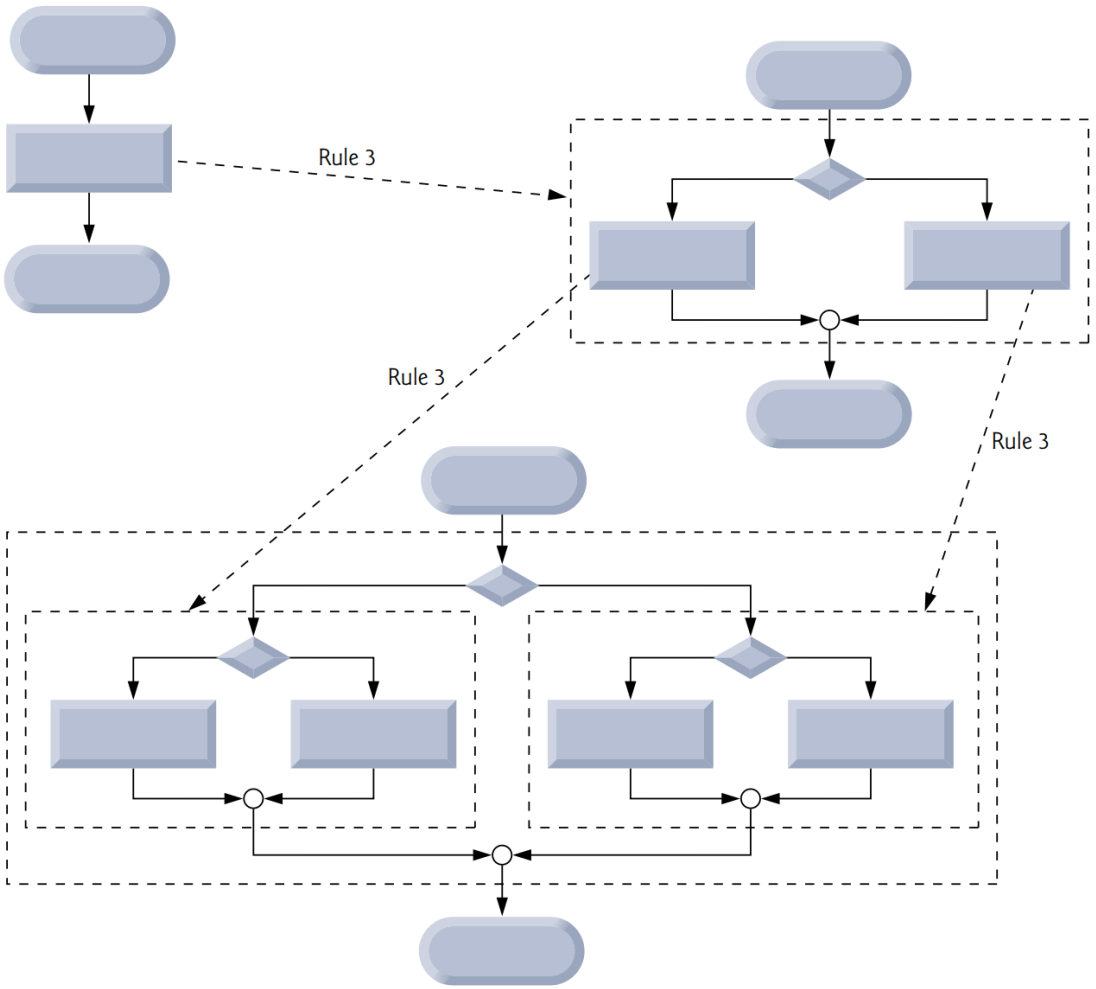


Fig. 4.21 | Applying Rule 3 of Fig. 4.18 to the simplest flowchart.

Summary

- All programs can be broken down into 3 controls
  - Sequence – handled automatically by compiler
  - Selection – `if`, `if…else` or `switch`
  - Repetition – `while`, `do…while` or `for`
    - Can only be combined in two ways
      - Nesting (rule 3)
      - Stacking (rule 2)
  - Any selection can be rewritten as an if statement, and any repetition can be rewritten as a while statement

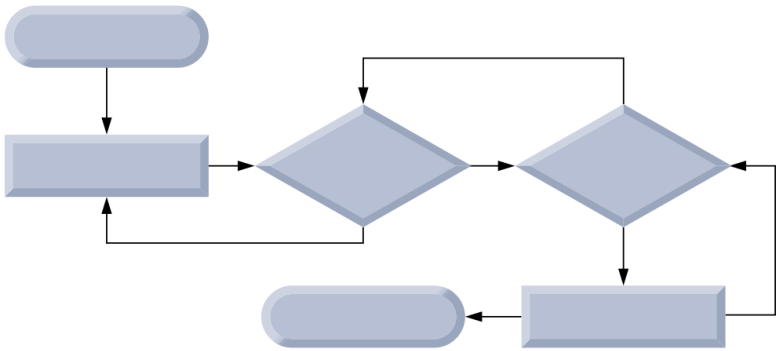


Fig. 4.23 | An unstructured flowchart.

Quiz

4.4 Find the error in each of the following code segments and explain how to correct it.

a)
```
x = 1;
while (x <= 10);
   ++x;
}
```

b)
```
for (double y = .1; y != 1.0; y += .1) {
   printf("%f\n", y);
}
```

c)
```
switch (n) {
   case 1:
      puts("The number is 1");
   case 2:
      puts("The number is 2");
      break;
   default:
      puts("The number is not 1 or 2");
      break;
   }
```

d) The following code should print the values 1 to 10.

```
n = 1;

while (n < 10) {
  printf("%d ", n++);
}
```

Quiz

4.5 Find the error in each of the following. (*Note:* There may be more than one error.)

a)
```
for (a = 25, a <= 1, a--); {
  printf("%d\n", a);
}
```

b) The following code should print whether a given integer is odd or even:

```
switch (value) {
   case (value % 2 == 0):
       puts("Even integer");
   case (value % 2 != 0):
       puts("Odd integer");
}
```

Quiz

c) The following code should calculate incremented salary after 10 years:

```
for (int year = 1; year <= 10; ++year) {
    double salary += salary * 0.05;
}

printf("%4u%21.2f\n", year, salary);
```

d)

```
for (double y = 7.11; y != 7.20; y += .01)
  printf("%7.2f\n", y);
```

e) The following code should output all multiples of 3 from 1 to 100:

```
for (int x = 3; x <= 100; x%3 == 0; x++ ) {
  printf("%d\n", x);
}
```

f)

```
x = 1;
while ( x <= 10 ) {
    printf("%d\n", x);
}
```

Quiz

4.6 State which values of the control variable x are printed by each of the following for statements:

a)
```
for (x = 20; x >= 3; x -= 3) {
  printf("%u\n", x);
}
```

b)
```
for (x = 7; x <= 27; x += 5) {
  printf("%u\n", x);
}
```

c)
```
for (x = 2; x <= 20; x += 4) {
  printf("%u\n", x);
}
```

d)
```
for (x = 30; x >= 15; x -= 6) {
  printf("%u\n", x);
}
```

e)
```
for (x = 22; x >= 2; x -= 5) {
  printf("%d\n", x);
}
```

Quiz

4.8 What does the following program do?

```c
#include <stdio.h>
int main(void)
{
   int x, i, j;
   // prompt user for input
   printf("%s", "Enter an integer in the range 1-20:");
   scanf("%d", &x); // read values for x
   for (i = 1; i <= x; i++) { // count from 1 to x
      for (j = 1; j <= x; j++) { // count from 1 to x
         if (j==i)
            printf("%c", '@'); // output @
         else
            printf("  ");
      } // end inner for
      printf("\n");
   } // end outer for
} // end of main
```

Quiz

4.36 What does the following program segment do?

```
int n = 4, a = 1;
   int i, c;
      for (i = 1; i <= n; i++) {

         for (c = 1; c <= i; c++) {

            printf("%d", a);
            a++;
         }
   printf("\n");
}
```

Quiz

4.24 Assume `i = 5`, `j = 7`, `k = 4` and `m = -2`. What does each of the following statements print?

a) `printf("%d", i == 5);`

b) `printf("%d", j!= 3);`

c) `printf("%d", i >= 5 && j < 4);`

d) `printf("%d", !m && k > m);`

e) `printf("%d", !k || m);`

f) `printf("%d", k - m < j || 5 - j >= k);`

g) `printf("%d", j + m <= i && !0);`

h) `printf("%d", !(j - m));`

i) `printf("%d", !(k > m));`

j) `printf("%d", !(j > k));`

Assignment 1

4.26 *(Calculating the Value of π)* Calculate the value of π from the infinite series

$$\pi = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} + \cdots$$

Print a table that shows the value of π approximated by one term of this series, by two terms, by three terms, and so on. How many terms of this series do you have to use before you first get 3.14? 3.141? 3.1415? 3.14159?

Write a program using for, switch, do...while

Assignment 2

Write a program to read value of N entered by user and print all Leap Years from 1 to N years. There are two conditions for leap year: 1- If year is divisible by 400 (for Century years), 2- If year is divisible by 4 and must not be divisible by 100 (for Non Century years).

Write a program using for, switch, do...while

Assignment 3

- Write a C program that allows the user to enter n number (n > 0 and must be entered first), calculate and print the average of all positive number.

Write a program using for, switch, do...while

Assignment 4

Write a C program to collect the temperature in the garden. Your program should:

a) input the temperature at 12 different times of the day. If the temperature is higher than 44.30, the message “Caution! High temperature” will be showed.

b) determine and display the average temperature in a day.

c) find and print the largest temperature in a day.

d) find and print the second largest temperature in a day. Use the function to complete this task.

Write a program using for, switch, do...while

Assignment 5

Write a program in C to find the sum of the series as follows

$$\overbrace{\underbrace{x}_{\text{1 term}} - x^3 + x^5 - x^7}^{\text{4 terms}} + x^9 - x^{11} + \cdots$$

with x and the number of terms are entered by user.

Write a program using for, switch, do...while

Assignment 6

Write a program using switch statement to calculate the weakly salary of

- Manager (codeID = 1) with a fixed salary 240 USD/day. Inputs (entered by user): number of working day.
- Hourly worker (codeID = 2) with 22 USD/hour for the first 40 hours and 1.5 times for overtime working. Inputs (entered by user): number of working hours in a week.
- Commission worker (codeID = 3) with 250 USD/week plus 7.2% of their gross weekly sale. Inputs (entered by user): weekly sale money
- Seller (codeID = 4) with a fixed incentive money for each selling item. Inputs (entered by user): a fixed incentive money and number of selling items.

Write a program using for, switch, do...while